

Datos Generales del Proyecto

Universidad:

Universidad Mariano Gálvez de Guatemala

Facultad:

Facultad de ingeniería en sistemas (sede de Boca del monte)

Carrera:

Ingeniería en sistemas de Información y ciencias de la computación

Curso:

Análisis de sistemas I

Período / Ciclo académico:

Primer semestre 2026

Fecha:

[10/06/2026]

Lugar:

Ciudad de Guatemala, Guatemala

Equipo de trabajo

Nombre completo	Carné	Correo institucional	Rol en el equipo
Elmer Yovani Rodríguez de Paz	7690-22-11804	erodriguezd10@mi umg.edu.gt	Scrum Master/Team
Freddy Roberto Orozco Alvarado	7690-22-16980	forozcoa2@miumg.edu.gt	Product Owner/Team

Asesor(a) / Catedrático(a):

Mélvín Cali

Fechas clave

Descripción	Fecha
Fecha de inicio del proyecto	20/02/2026
Fecha de entrega acordada	05/06/2026
Fecha efectiva de entrega	10/06/2026

Datos generales del proyecto

Información básica e identificadores del proyecto.

Campo	Valor
Nombre del proyecto	Sistema La Esperanza — Plataforma de Gestión Agrícola
Tipo de proyecto	Desarrollo de software (investigación aplicada)
Modalidad	Equipos (Proyecto con varias fases de entrega)
Curso	Análisis de sistemas I
Carrera	Ingeniería en Sistemas de Información y ciencias de la computación
Versión actual	1.1.1
Fecha de inicio	[20/02/2026]
Fecha de entrega	[10/06/2026]
Repositorio	https://github.com/Gasel004/PROTOTIPOAS.git
Estado	Finalizado (release 1)
URL	https://sistemalaesperanza.xyz

Resumen ejecutivo

El Sistema La Esperanza es una plataforma web que digitaliza y formaliza el proceso de comercialización agrícola directa entre productores y compradores, incorporando a las asociaciones de agricultores como entidades organizadoras y de garantía, a su vez permite una vista sin el inicio de sesión respectivo para mayor atracción a los compradores. La solución elimina la dependencia de intermediarios, reduce las asimetrías de información y provee trazabilidad inicio-fin de cada operación. Cubre siete módulos funcionales: Autenticación con JWT y roles, Publicaciones de ofertas, Negociaciones privadas, Mensajería, Entregas con doble confirmación, Pagos y Notificaciones. Está construida con React 18 + Vite en el frontend, Node.js 20 + Express + Prisma ORM en el backend y PostgreSQL 16 como motor de persistencia, orquestados mediante Docker y Nginx.

Planteamiento del problema

En la cadena de comercialización agrícola tradicional, los productores dependen de intermediarios que reducen sus márgenes de ganancia, mientras que los compradores enfrentan asimetrías de información sobre disponibilidad real, calidad y precio de los productos. Adicionalmente, la ausencia de un canal directo y de un mecanismo formal de registro provoca falta de control en las operaciones, incrementando así el riesgo de impago y dificulta la rendición de cuentas por parte de las asociaciones que agrupan a los productores. Se hace necesaria, por tanto, una plataforma digital que conecte de forma directa a productores, compradores y asociaciones, ofreciendo un canal confiable, transaccional y trazable para las operaciones agrícolas.

Oportunidad / justificación

La oportunidad consiste en aprovechar la creciente penetración de conectividad y dispositivos móviles en zonas agrícolas para desarrollar una plataforma web que conecte de manera directa, transaccional y trazable a productores, compradores y asociaciones. La justificación del proyecto se sustenta en: (i) la necesidad expresada por las asociaciones de formalizar su gestión interna; (ii) la demanda de compradores que requieren un canal único de oferta verificable; (iii) la disponibilidad de un stack tecnológico maduro, gratuito y reproducible (React, Node.js, Prisma, PostgreSQL, Docker); y (iv) la viabilidad del alcance dentro de un semestre académico.

Descripción de la solución propuesta

Sistema La Esperanza es una plataforma web (SPA) que digitaliza y formaliza la comercialización agrícola directa entre productores y compradores, con las asociaciones como entidades organizadoras y de garantía. La solución se construye sobre tres roles diferenciados (productor, comprador y asociación) y siete módulos funcionales: Autenticación, Publicaciones, Negociaciones, Mensajería, Entregas, Pagos y Notificaciones. La arquitectura combina un frontend en React 18 + Vite, un backend en Node.js 20 + Express + Prisma, una base de datos PostgreSQL 16 y una capa de orquestación con Docker + Nginx, lo que garantiza portabilidad y reproducibilidad del entorno.

Objetivos

Objetivo general

Desarrollar una plataforma web responsiva para que digitalice y gestione la comercialización agrícola directa entre productores, compradores y asociaciones, ofreciendo trazabilidad, comunicación y registro transaccional de las operaciones.

Objetivos específicos

- Implementar un módulo de autenticación con registro, login y cierre de sesión basado en JSON Web Tokens (JWT) y roles diferenciados (productor, comprador, asociación).
- Desarrollar toda la documentación respectiva previa a la codificación del sistema, incluyendo los diagramas respectivos, los manuales etc..
- Integrar un sistema de Mensajería interna asociado a cada negociación para facilitar la comunicación directa entre las partes.
- Implementar el módulo de Entregas con confirmación doble (entrega por parte del productor y recepción por parte del comprador) para asegurar trazabilidad.
- Aplicar la metodología SCRUM con ayuda de plataformas externas durante el proceso de desarrollo del sistema, desde su inicio hasta el final.
- Uso correcto de las diferentes técnicas para la respectiva recolección de datos, para obtener los requerimientos previos para el sistema.

Beneficios esperados

Beneficiario	Beneficio esperado
Productor	Acceso directo a compradores, eliminación de intermediarios, trazabilidad de sus publicaciones, historial de negociaciones y control de entregas y pagos.
Comprador	Catálogo único de publicaciones con búsqueda y filtros, comunicación directa con el productor, historial de operaciones y certificación de la entrega recibida.
Asociación	Visibilidad agregada de las operaciones de sus productores asociados, panel de indicadores, supervisión de publicaciones y respaldo institucional en las transacciones.
Equipo de desarrollo	Construcción de un portafolio técnico reproducible con un stack moderno (React + Node + Prisma + Docker) y entregables académicos formales (documentación, pruebas, cobertura).

Funcionalidades entregadas

Módulo / Pantalla	Descripción	Rol(es)	Estado
Registro y Login	Autenticación con JWT por rol	Todos	Completa

	(productor, comprador, asociación). Registro crea perfil específico por rol.		
Dashboard por rol	Vistas diferenciadas: KPIs del productor (publicaciones, negociaciones, ingresos); KPIs del comprador (negociaciones, entregas, pagos); resumen de asociación (miembros, actividad).	Todos	Completa
Publicaciones	Listado público de ofertas agrícolas con filtros por departamento, municipio, categoría. Vista de detalle.	Todos	Completa
Mis Publicaciones	Gestión personal: crear, editar, cambiar estado (activa/pausada/cerrada), eliminar.	Productor	Completa
Crear / Editar Publicación	Formulario con subida de imagen, selección de producto del catálogo, precio, cantidad, fecha de cosecha, ubicación.	Productor	Completa
Detalle de Publicación	Vista completa de la publicación con opción de iniciar negociación para compradores.	Todos	Completa
Negociaciones	Listado de negociaciones activas e historial. Canal privado productor ↔ comprador.	Productor / Comprador	Completa
Detalle de Negociación	Chat de mensajería integrado, cambio de estado de la negociación, creación de entrega y pago asociados.	Productor / Comprador	Completa
Entregas	Registro de entregas con	Productor /	Completa

		confirmación doble (productor y comprador deben confirmar independientemente).	Comprador	
Pagos		Registro y seguimiento de pagos por negociación. Métodos de pago, referencia, monto, estado.	Productor / Comprador	Completa
Perfil		Edición de datos personales por rol (nombre, teléfono, ubicación, descripción). Cambio de contraseña.	Todos	Completa
Miembros de Asociación		Gestión de productores afiliados: ver lista, suspender temporal o definitivamente, reactivar.	Asociación	Completa
Catálogo de Productos		Administración del catálogo de productos agrícolas: crear, editar, activar/desactivar.	Asociación	Completa
Notificaciones		Alertas automáticas del sistema entregadas por evento. Marcar como leída (individual o todas).	Todos	Completa

Reglas del negocio implementadas

- La autenticación usa número de teléfono (formato 4256-1234) como identificador único, no correo electrónico.
- El registro crea automáticamente un perfil específico según el rol: productor, comprador o asociación.
- Solo los productores pueden publicar ofertas de venta y gestionar sus publicaciones.
- Solo los compradores pueden iniciar una negociación a partir de una publicación activa.
- Las negociaciones son canales privados: únicamente el productor y el comprador involucrados pueden ver los mensajes.

- La confirmación de entregas es doble: tanto el productor como el comprador deben confirmar para que la entrega quede completada.
- Las asociaciones pueden suspender (temporal o definitivamente) a productores miembros, con motivo registrado.
- Solo las asociaciones administran el catálogo de productos (creación y edición); los productores seleccionan del catálogo al publicar.
- Las contraseñas se almacenan con hash bcrypt (factor de coste 10), nunca en texto plano.
- Los tokens JWT tienen expiración configurable (por defecto 7 días) e incluyen id, teléfono y rol en el payload.
- El acceso a rutas protegidas requiere Bearer Token; sin token o con token inválido se retorna HTTP 401.
- El acceso a acciones fuera del rol retorna HTTP 403.

Alcance funcional cubierto sobre las reglas del negocio

- Un usuario solo puede tener un rol (productor, comprador o asociación).
- Un teléfono solo puede estar registrado una vez.
- El DPI del productor y el NIT del comprador son únicos.
- La contraseña debe tener mínimo 8 caracteres y se almacena hasheada con bcrypt (10 rounds).
- El teléfono debe cumplir el formato 0000-0000 (regex).
- Una negociación solo puede existir sobre una publicación en estado "activa".
- La cantidad solicitada en una negociación no puede superar la cantidad disponible de la publicación.
- La negociación requiere confirmación independiente de productor y comprador para pasar a "aceptada".
- Solo el productor y el comprador involucrados en una negociación pueden verla/mensajearla.
- La asociación puede ver todas las negociaciones y entregas de sus miembros.
- La publicación se desactiva automáticamente al alcanzar la fecha de cosecha (estado "vencida").
- La confirmación de entrega es independiente por cada parte y ambas deben confirmar para completar la entrega.

- Eliminación lógica de publicaciones (campo "eliminada" en BD, no borrado físico).
- Un productor o comprador suspendido no puede iniciar sesión ni operar.
- Solo la asociación tiene acceso a la bitácora de auditoría y a las estadísticas globales.

Requisitos no funcionales cubiertos

Requisito	Descripción	Estado
Seguridad (autenticación)	JWT Token con expiración. Contraseñas con bcrypt. Control de acceso por rol en cada endpoint.	Cumplido
Seguridad (CORS)	CORS configurado para aceptar únicamente los orígenes autorizados (variable FRONTEND_URL).	Cumplido
Escalabilidad horizontal	Arquitectura en contenedores Docker permite replicar servicios de forma independiente.	Cumplido
Portabilidad	Docker Compose orquesta todos los servicios (DB, backend, frontend/nginx) con un solo comando.	Cumplido
Timeout de requests	El servidor aplica timeout de 30 segundos por petición, retornando HTTP 503 si se supera.	Cumplido
Lazy loading (frontend)	Todas las páginas se cargan de forma diferida con React.lazy + Suspense para mejorar el tiempo de carga inicial.	Cumplido
Accesibilidad básica	Región ARIA live para anuncios de pantalla, scroll to top en navegación, semántica HTML5.	Parcial
HTTPS en producción	Requerido por el pliego de entrega. Configuración pendiente a nivel de VPS/nginx con certificado SSL.	Pendiente VPS

Elementos excluidos

Funcionalidad	Motivo	Recomendación futura
---------------	--------	----------------------

HTTPS automático en VPS	Fuera del alcance del entregable de código; se configura a nivel de infraestructura con Certbot + nginx en el servidor real.	Agregar servicio certbot al docker-compose.yml o configurar nginx en host con Let's Encrypt.
Sistema de calificaciones UI	El modelo de datos y el backend están implementados (modelo Calificación con moderación de asociación), pero la interfaz de usuario para emitir y visualizar calificaciones no fue incluida en el frontend.	Implementar pantalla de calificación al completar una negociación.
Notificaciones en tiempo real	Las notificaciones se generan en el servidor (BD) pero no existe canal WebSocket/SSE para entrega en tiempo real.	Agregar Socket.io o SSE al backend para push de notificaciones.
Panel de administración global	No existe un rol de superadmin con vista global del sistema.	Implementar rol admin con dashboards agregados.
Pagos en línea	El módulo de pagos registra pagos manuales (referencia bancaria), no integra pasarela de pago.	Integrar Stripe, PayPal o pasarela local según mercado objetivo.

Escalabilidad

- Arquitectura de microservicios contenerizada: 3 servicios independientes (db, backend, frontend) escalables individualmente.
- Volúmenes Docker nombrados para persistencia (pgdata, uploads_data), desacoplados del ciclo de vida de los contenedores.
- Multi-stage build en el backend (imagen final más liviana).
- Singleton de Prisma Client para evitar conexiones duplicadas a la BD.

Entregables

Código fuente y repositorio

URL del repositorio	https://github.com/Gasel004/PROTOTIPOAS
Rama principal	main
Versión etiquetada	v1.1.1 (git tag)
Rama de desarrollo	Fredy, yovani. Una por participante
Estructura del repo	monorepo: /frontend, /backend, /Documentación, docker-compose.yml

Documentación

La siguiente tabla enumera los entregables formales de la versión 1.0.0, su descripción, formato y criterio de aceptación.

#	Entregable	Descripción	Formato
Código fuente frontend	Implementación completa de la SPA en React + Vite.	Repositorio Git	npm run build sin errores; npm run dev arranca en :5173.
Código fuente backend	Implementación completa de la API REST en Node + Express + Prisma.	Repositorio Git	npm run start arranca en :3000; /api/v1/health responde {status:ok}.
Esquema Prisma y migraciones	Definición del modelo relacional y migraciones versionadas.	schema.prisma + migrations/	prisma migrate deploy aplica sin errores en BD limpia.
Script de seed con datos de prueba	Datos iniciales (3 cuentas, productos, publicaciones de ejemplo).	backend/prisma/seed.js	npm run db:seed puebla la BD con 3 cuentas 4000-0001/2/3.
docker-compose.yml + Dockerfiles	Orquestación de los 3 servicios y de Nginx.	Raíz del repo	docker compose up --build -d deja los 3 servicios healthy.
Manual de instalación	Guía paso a paso de	INSTRUCCIONES.txt	Un evaluador externo puede levantar el sistema siguiendo

	instalación, despliegue y operación.		solo este archivo.
Documento de Datos Generales	Información institucional y resumen ejecutivo.	Datos_Generales_LaEsperanza.docx	Documento firmado por el equipo.
Documento de Alcance	El presente documento.	Documento_Alcanze_LaEsperanza.docx	Aprobado por el asesor(a) y la coordinación.
Diccionario de datos	Definición detallada de cada entidad, atributo y relación.	Documentación/ Diccionario_Datos_LaEsperanza.docx	Cubre el 100 % de los modelos Prisma.
Documentación de API REST	Especificación de los 12 grupos de rutas, payloads y respuestas.	Documentación/ Documentación_API_LaEsperanza.docx	Cada endpoint documentado con ejemplo de request y response.
Plan de pruebas	Estrategia, alcance, criterios de entrada/salida y entorno.	Documentación/ Plan_Pruebas_LaEsperanza.docx	Plan firmado por el QA Lead y el asesor(a).
Casos de prueba	Casos funcionales, de integración y de aceptación con su evidencia.	Documentación/ Casos_Prueba_LaEsperanza.docx	≥ 30 casos ejecutados; ≥ 90 % con resultado PASS.
Matriz de trazabilidad	Tabla Requisito ↔ Diseño ↔ Código ↔ Prueba.	Documentación/ Matriz_Trazabilidad_LaEsperanza.docx	Cobertura ≥ 95 % sobre RF y RNF.
Release notes	Notas de la versión 1.0.0: cambios, mejoras, correcciones.	Documentación/ Release_Notes_LaEsperanza.docx	Publicadas en el repositorio.
README.md	Puerta de entrada al proyecto:	README.md	Permite a un nuevo integrante entender el sistema en < 10 minutos.

	descripción, quick-start y enlaces.		
--	---	--	--

Instrucciones de compilación y ejecución

Opción A — Con Docker (recomendada para producción):

- `git clone https://github.com/Gasel004/PROTOTIPOAS.git && cd PROTOTIPOAS`
- `cp .env.example .env # configurar variables de entorno`
- `docker compose up --build -d`
- `docker compose exec backend npx prisma migrate deploy`
- `docker compose exec backend npx prisma db seed`
- Acceso: `http://localhost:8081` (frontend) · `http://localhost:3000/api/v1` (API)

Opción B — Desarrollo local (sin Docker):

- Requisitos: Node.js 20 LTS, PostgreSQL 16
- Backend: `cd backend && npm install && cp ../.env.example .env → editar DATABASE_URL → npx prisma migrate dev → npm run dev`
- Frontend: `cd frontend && npm install && echo VITE_API_URL=http://localhost:3000/api/v1 > .env.local → npm run dev`

Documentación técnica incluida

Documento	Formato	Descripción
Arquitectura, Componentes y Patrones	DOCX	Patrones arquitectónicos, diagrama de componentes y decisiones de diseño del sistema.
Documentación API REST	DOCX	Especificación de los 35+ endpoints: método, ruta, autenticación, parámetros, respuestas.
Casos de Prueba	DOCX	37 casos de prueba documentados (unitarias, integración, sistema,

		aceptación).
Evidencias de Cobertura	DOCX	Capturas y resultados de cobertura de pruebas automatizadas.
Pruebas Automatizadas (código)	JS	Archivo .test.js con ejemplos de pruebas automatizadas con Jest + Supertest.
Gitflow	ODT	Estrategia de ramas, flujo de trabajo y convenciones de commits.
Guía de Instalación	ODT	Instalación completa con Docker y sin Docker para Windows y Ubuntu.
Guía de Entorno de Desarrollo	ODT	Configuración del ambiente de desarrollo local.
Guía de Lanzamiento	ODT	Proceso de despliegue en VPS, configuración de producción.
Historial de Tareas	ODT	Registro cronológico de tareas y decisiones del proyecto.
Infraestructura del Sistema	ODT	Descripción de la infraestructura: contenedores, red, volúmenes, dependencias.
Matriz de Trazabilidad	ODT	Relación RF ↔ Implementación ↔ Casos de prueba.
Plan de Pruebas	ODT	Estrategia de QA, tipos de prueba, criterios de aceptación.
Release Notes v1.0.0	ODT	Novedades, cambios y notas de la versión inicial.
Script de Creación de Base de Datos	ODT	Script SQL de referencia para creación de la base de datos.

Vista de la Base de Datos	ODT	Vistas SQL y estructura de la base de datos documentada.
Diagrama de Arquitectura	PNG	Diagrama visual de la arquitectura del sistema generado con draw.io.
Documento de Entrega Final (este archivo)	DOCX	Documentación consolidada de entrega.

Otros entregables

Artefacto	Descripción	Estado
docker-compose.yml	Orquestación completa de los 3 servicios (db, backend, frontend).	Incluido en repo
.env.example	Plantilla de variables de entorno con valores por defecto documentados.	Incluido en repo
Migración de BD	prisma/migrations/20260603123835_init/migration.sql — script SQL generado por Prisma con toda la estructura.	Incluido en repo
Prisma Schema	backend/prisma/schema.prisma — definición completa de los 13 modelos.	Incluido en repo
Seed de datos	backend/prisma/seed.js — datos iniciales de prueba para desarrollo.	Incluido en repo
Accesos y credenciales	Variables del .env.example. Credenciales de producción del VPS.	[COMPLETA R: Pendiente VPS]
Backup inicial de BD	Backup del estado inicial de la base de datos en producción.	[COMPLETA R: Pendiente VPS]
Diagramas0	Todos los diagramas relacionados con el sistema, antes ya realizados, pero con los cambios se realizaron	Completos

	actualizaciones de los mismos	
--	-------------------------------	--

Alcance del proyecto

Alcance funcional

El sistema cubre los siete módulos descritos y tres roles principales y uno secundario:

- Módulo de Autenticación: registro, login y logout con JWT por rol.
- Módulo de Publicaciones: crear, listar, editar y eliminar ofertas.
- Módulo de Negociaciones: apertura, seguimiento y cierre de tratos directos.
- Módulo de Mensajería: chat privado asociado a cada negociación.
- Módulo de Entregas: registro de entregas con confirmación doble.
- Módulo de Pagos: registro y seguimiento del estado de pago.
- Módulo de Notificaciones: alertas automáticas sobre eventos del sistema.
- Roles: Productor, Comprador y Asociación (entidad organizadora) y el visitante sin sesión.

Alcance técnico

Stack tecnológico implementado:

- Frontend: React 18 + Vite 5 + React Router v6 + Zustand 4 + Axios 1.7 + lucide-react.
- Backend: Node.js 20 LTS + Express 4 + Prisma ORM 5 + JWT + Multer + bcryptjs.
- Base de datos: PostgreSQL 16.
- Infraestructura: Docker, Docker Compose y Nginx como proxy inverso.

Fuera del alcance

- No es un e-commerce con pasarela de pago real: el módulo de Pagos es de registro y seguimiento, no procesa transacciones bancarias.
- No incluye facturación electrónica ni integración con autoridades tributarias.
- No contempla aplicación móvil nativa (iOS/Android); es una aplicación web responsive.
- No integra sistemas externos de logística, transporte o geolocalización en tiempo real.
- No implementa un sistema de reputación avanzado distinto a la Calificación básica entre usuarios.
- No incluye multi-tenancy ni despliegue en múltiples regiones.

Supuestos

Los siguientes supuestos son válidos durante toda la ejecución del proyecto. Su ruptura obliga a una re-planificación documentada.

- Conectividad estable a PostgreSQL 16 desde el backend dentro de la red local.

- Disponibilidad de un navegador moderno (Chrome, Firefox, Edge en sus dos últimas versiones mayores) para el acceso a la SPA.
- Los dispositivos de los productores cuentan con cámara para tomar y subir fotografías de los productos ofertados.
- Los datos de seed son representativos del dominio agrícola local y suficientes para los flujos de prueba end-to-end.
- Los números telefónicos siguen el formato 4000-0000 (4 dígitos + 4 dígitos) para fines de prueba y validación de los seed accounts.
- El equipo de desarrollo tiene acceso a un entorno con Docker Desktop o alternativa compatible con Docker Compose v2.
- El asesor(a) realiza al menos una revisión formal por hito del cronograma.

Fuera del alcance (en exclusión)

Las siguientes funcionalidades quedan explícitamente fuera del alcance de la versión 1.0.0 del Sistema La Esperanza. Su inclusión corresponderá a versiones futuras o a proyectos derivados.

1. E-01. Pasarela de pago real (Stripe, PayPal, u otras). El módulo de Pagos se limita al registro y seguimiento del estado del pago; no procesa transacciones bancarias.
2. E-02. Facturación electrónica con SAT u otros organismos tributarios.
3. E-03. Aplicación móvil nativa (iOS / Android). El sistema se entrega como una SPA web responsiva, no como app nativa.
4. E-04. Geolocalización en tiempo real del transporte o de los actores.
5. E-05. Integración con APIs externas de tipo de cambio, clima o mercados internacionales.
6. E-06. Logística de transporte: la aplicación únicamente registra el lugar y la fecha de entrega acordados, sin gestionar flotas o rutas.
7. E-07. Módulo de inventario o stock multi-bodega.
8. E-08. Firma electrónica legalmente vinculante sobre los documentos transaccionales.
9. E-09. Multi-idioma: la interfaz y la documentación se entregan exclusivamente en español.
10. E-10. Multi-tenancy: un único espacio comercial por despliegue; no se admite múltiples clientes conviviendo en la misma instancia.
11. E-11. Análisis avanzado / BI / machine learning sobre los datos transaccionales.
12. E-12. Notificaciones push nativas en dispositivos móviles (solo notificaciones in-app dentro de la SPA).

Actores del sistema

Actor	Descripción	Permisos clave
Productor	Persona natural o jurídica que ofrece productos agrícolas a través de	Crear y administrar publicaciones, responder negociaciones, confirmar

	publicaciones.	entregas, gestionar su perfil.
Comprador	Persona natural o jurídica interesada en adquirir productos publicados.	Explorar publicaciones, abrir negociaciones, enviar mensajes, confirmar recepción, registrar pagos.
Asociación	Entidad organizadora que agrupa productores, da respaldo y supervisa las operaciones.	Gestionar productores asociados, validar publicaciones, supervisar negociaciones y ver indicadores globales.
Comprador sin sesión (visitador)	Visitante ajeno que accede únicamente al catalogo de productos disponibles.	Ninguno, simplemente ve el catalogo, no le es posible realizar pedidos

Requisitos técnicos cumplidos

Lenguajes de programación

Lenguaje	Versión	Capa	Uso
JavaScript (ES2022+)	ES2022	Frontend / Backend	Lenguaje principal en ambas capas
JSX	React 18	Frontend	Sintaxis de componentes React
SQL	PostgreSQL 16	Base de datos	Queries generadas por Prisma ORM
CSS	CSS3	Frontend	Estilos globales (index.css ~94 KB)

Frameworks y bibliotecas

Capa	Tecnología	Versión	Propósito
Frontend	React	18.x	Construcción de la interfaz de usuario basada en componentes.
Frontend	Vite	5.x	Empaquetador y servidor de desarrollo rápido.
Frontend	React Router	v6	Enrutamiento declarativo del lado cliente.

Frontend	Zustand	4.x	Gestión ligera de estado global.
Frontend	Axios	1.7.x	Cliente HTTP para consumo de la API REST.
Frontend	lucide-react	última	Set de íconos vectoriales ligeros para la UI.
Backend	Node.js	20 LTS	Entorno de ejecución del servidor.
Backend	Express	4.x	Framework HTTP y middleware.
Backend	Prisma ORM	5.x	Mapeo objeto-relacional y migraciones.
Backend	JSON Web Token	9.x	Autenticación stateless por tokens.
Backend	Multer	2.x	Carga y procesamiento de archivos (imágenes de productos).
Backend	bcryptjs	2.x	Hashing de contraseñas.
Base de datos	PostgreSQL	16	Motor relacional principal.
Infraestructura	Docker	última	Contenerización de servicios.
Infraestructura	Docker Compose	v2	Orquestación multi-contenedor.
Infraestructura	Nginx	última	Proxy inverso y servidor de archivos estáticos.
Pruebas	Jest	29.x	Framework de pruebas unitarias e integración (configurado).
Pruebas	Supertest	7.x	Pruebas HTTP sobre endpoints Express (configurado).

Infraestructura tecnológica

Componente	Tecnología	Detalles
Contenedor de base de datos	PostgreSQL 16 Alpine	Expuesto en puerto 5433:5432. Volumen persistente pgdata. Healthcheck configurado.
Contenedor de backend	Node.js 20 + Express	Expuesto en puerto 3000. Depende de DB con condición service_healthy.
Contenedor de frontend	Nginx + React build	Expuesto en puerto 8081:80. Sirve el build estático de Vite.
Servidor VPS	[COMPLETAR: Dirección IP del VPS]	[COMPLETAR: Proveedor y specs del VPS]
Dominio / HTTPS	[COMPLETAR: Dominio configurado]	Certificado SSL con Let's Encrypt (Certbot) — pendiente de configuración en VPS.

Cumplimiento de estándares de seguridad

Estándar / Práctica	Implementación	Estado
Autenticación (OWASP A07)	JWT firmado con HS256. Expiración configurable. Token requerido en encabezado Authorization: Bearer.	Cumplido
Almacenamiento de credenciales (OWASP A02)	Contraseñas hasheadas con bcrypt factor 10. Hash no expuesto en ninguna respuesta de API.	Cumplido
Control de acceso por roles (OWASP A01)	Middleware soloRoles() en cada ruta protegida. Tres roles: productor, comprador, asociación.	Cumplido
Configuración segura (OWASP A05)	CORS restringido a orígenes explícitos. Variables de entorno	Cumplido

	en .env (nunca en código fuente).	
HTTPS / TLS	Pendiente de configuración en VPS. El sistema está diseñado para correr detrás de nginx con TLS.	Pendiente VPS
Inyección SQL	Prisma ORM usa queries parametrizadas automáticamente. No hay SQL crudo en el código.	Cumplido
Exposición de datos sensibles	password_hash excluido de todas las respuestas con desestructuración explícita en DTOs.	Cumplido

Módulos del sistema

A continuación, se describen los siete módulos cubiertos por la primera versión entregable del sistema, con sus endpoints principales bajo el prefijo /api/v1.

Módulo	Descripción breve	Endpoints principales
Autenticación	Registro, login, logout y validación de sesión con JWT por rol.	POST /auth/register · POST /auth/login · GET /auth/me · POST /auth/logout
Publicaciones	Gestión de ofertas de productos agrícolas (crear, listar, editar, eliminar).	GET /publicaciones · POST /publicaciones · PUT /publicaciones/:id · DELETE /publicaciones/:id
Negociaciones	Canal privado productor ↔ comprador para acordar condiciones.	GET /negociaciones · POST /negociaciones · PATCH /negociaciones/:id · POST /negociaciones/:id/cerrar
Mensajería	Chat interno ligado a cada negociación.	GET /negociaciones/:id/mensajes · POST /negociaciones/:id/mensajes
Entregas	Registro de entregas con confirmación doble (productor y comprador).	POST /entregas · POST /entregas/:id/confirmar · GET /entregas

Pagos	Registro y seguimiento del estado de pago de cada negociación.	POST /pagos · GET /pagos · PATCH /pagos/:id
Notificaciones	Panel de alertas automáticas generadas por el sistema.	GET /notificaciones · PATCH /notificaciones/:id/leida · POST /notificaciones/leer-todas

Riesgos identificados

Matriz de riesgos con probabilidad e impacto cualificados como Bajo (B), Medio (M) o Alto (A), junto con la estrategia de mitigación propuesta.

#	Riesgo	Probabilidad	Impacto	Estrategia de mitigación
RI-01	Cambios de requisitos a mitad del ciclo	M	A	Congelar el alcance tras el hito de Diseño; canalizar cualquier cambio vía control de versiones del documento de Alcance.
RI-02	Atraso o indisponibilidad de integrantes del equipo	M	M	Rotación de roles en pares (frontend/backend/QA); actas de seguimiento semanales; repositorio compartido desde el día 1.
RI-03	Incompatibilidad de versiones de Node.js en el equipo	B	A	Fijar Node 20 LTS en .nvmrc y en la imagen Docker; script de verificación de versión en INSTRUCCIONES.txt.
RI-04	Fallo de Prisma al regenerar el cliente	B	A	Documentar paso a paso prisma generate y prisma migrate dev; backup del cliente generado en el repositorio.
RI-05	Problemas de certificados o SSL en PostgreSQL 16	B	M	Despliegue académico sobre LAN sin TLS; documentar el procedimiento para habilitar SSL en producción como mejora futura.

R-I06	Conflictos de puertos (3000 / 5432 / 80 / 5173)	M	M	Variables de entorno para todos los puertos; sección de troubleshooting en INSTRUCCIONES.txt.
RI-07	Capacidad de almacenamiento para imágenes de productos	M	B	Límite de 5 MB por imagen validado en backend; limpieza periódica de uploads/ documentada en INSTRUCCIONES.txt.
RI-08	Dependencia de Nginx como servicio externo	B	M	Nginx se ejecuta dentro del stack Docker; en modo local, el frontend se sirve por Vite y el backend por Node.
RI-09	Pérdida de archivos por ausencia de respaldos	B	A	Repositorio Git como única fuente de verdad; política de commits pequeños y frecuentes; etiqueta de release por hito.
RI-10	Cambios en normativa universitaria durante el ciclo	B	M	Revisión quincenal de comunicados oficiales; canal directo con la coordinación para ajustes de calendario.

Ambientes para el despliegue

Entorno de desarrollo

Aspecto	Detalle
Sistema Operativo	Linux Mint / Ubuntu 22.04 LTS
Runtime backend	Node.js 20 LTS + npm
Base de datos	PostgreSQL 16 local (puerto 5432)
Servidor de desarrollo	nodemon (backend) + Vite dev server (frontend, puerto 5173)
ORM / Migraciones	Prisma Migrate (prisma migrate dev)
Control de versiones	Git + Gitflow (ramas main, develop, elmer, fredy)
Editor de código	Cursor, Zed, VSCODE
Variables de entorno	.env local con DATABASE_URL apuntando a localhost:5432

Entorno de pruebas

Aspecto	Detalle
Plataforma	Docker Compose local (mismo docker-compose.yml que producción)
URL de frontend	http://localhost:8081
URL de API	http://localhost:3000/api/v1
Base de datos	Contenedor laesperanza-db (PostgreSQL 16-alpine) con datos de seed
Datos de prueba	prisma/seed.js — carga datos iniciales de productores, compradores y asociaciones de ejemplo
Framework de pruebas	Jest 29 + Supertest 7 para pruebas de integración de endpoints
Ejecución de pruebas	cd backend && npm test / npm test -- --coverage
Herramienta manual	Postman / colección de API documentada

Entorno para producción

Aspecto	Detalle
Proveedor VPS	[COMPLETAR: Proveedor VPS (DigitalOcean, Linode, etc.)]
Dirección IP	[COMPLETAR: IP pública del VPS]
Sistema Operativo	[COMPLETAR: SO del VPS (ej. Ubuntu 22.04 LTS)]
Dominio	[COMPLETAR: Dominio configurado]
URL frontend (HTTPS)	[COMPLETAR: https://dominio.com]
URL API (HTTPS)	[COMPLETAR: https://dominio.com/api/v1 o

	https://api.dominio.com]
Certificado SSL	Let's Encrypt via Certbot — [PENDIENTE: configurar en servidor]
Puertos abiertos	80 (HTTP → redirect), 443 (HTTPS), 22 (SSH)
Orquestación	Docker Compose v2 — mismos 3 contenedores: db, backend, frontend
Persistencia de datos	Volumen Docker pgdata para datos de PostgreSQL
Control de acceso SSH	[COMPLETAR: Acceso via clave pública SSH. Usuario: configurado en VPS]

Análisis y diseño

Requerimientos funcionales (RF)

ID	Descripción	Actor(es)	Estado
RF-01	El sistema permite registrar usuarios con rol productor, comprador o asociación.	Todos	Implementado
RF-02	El sistema autentica usuarios por número de teléfono y contraseña.	Todos	Implementado
RF-03	El sistema genera un token JWT al iniciar sesión, válido por 7 días.	Todos	Implementado
RF-04	El productor puede crear publicaciones de oferta de productos agrícolas con imagen.	Productor	Implementado
RF-05	El productor puede editar, pausar, cerrar y eliminar sus publicaciones.	Productor	Implementado
RF-06	Todos los usuarios pueden explorar el catálogo de publicaciones con filtros.	Todos	Implementado

RF-07	El comprador puede iniciar una negociación privada con un productor.	Comprador	Implementado
RF-08	El sistema mantiene un canal de mensajería privado por negociación.	Productor/ Comprador	Implementado
RF-09	Los participantes pueden cambiar el estado de la negociación (aceptar, rechazar, cancelar).	Productor/ Comprador	Implementado
RF-10	El sistema requiere confirmación doble para registrar una entrega como completada.	Productor/ Comprador	Implementado
RF-11	El sistema permite registrar pagos con método, referencia, monto y estado.	Productor/ Comprador	Implementado
RF-12	El sistema genera notificaciones automáticas por eventos relevantes.	Todos	Implementado
RF-13	La asociación puede ver y gestionar la lista de productores afiliados.	Asociación	Implementado
RF-14	La asociación puede suspender (temporal o definitivamente) o reactivar productores.	Asociación	Implementado
RF-15	La asociación administra el catálogo de productos agrícolas.	Asociación	Implementado
RF-16	El usuario puede ver y editar su perfil, incluyendo cambio de contraseña.	Todos	Implementado
RF-17	El dashboard muestra KPIs diferenciados según el rol del usuario autenticado.	Todos	Implementado

Requerimientos no funcionales (RNF)

ID	Descripción	Estado
----	-------------	--------

RNF-01	El sistema debe ejecutarse en contenedores Docker para garantizar portabilidad.	Cumplido
RNF-02	El sistema debe operar bajo HTTPS en el entorno de producción.	Pendiente VPS
RNF-03	Las contraseñas deben almacenarse con hash irreversible (bcrypt, factor ≥ 10).	Cumplido
RNF-04	El sistema debe responder en menos de 30 segundos por petición (timeout configurado).	Cumplido
RNF-05	El frontend debe usar lazy loading de páginas para optimizar la carga inicial.	Cumplido
RNF-06	El acceso a recursos protegidos debe validarse mediante JWT en cada petición.	Cumplido
RNF-07	La base de datos debe persistir datos entre reinicios del contenedor mediante volúmenes Docker.	Cumplido
RNF-08	El sistema debe soportar los tres roles de usuario (productor, comprador, asociación) con permisos diferenciados.	Cumplido
RNF-09	El CORS debe restringirse a los orígenes autorizados definidos en variables de entorno.	Cumplido
RNF-10	El código fuente debe seguir el estándar de módulos ES (import/export) en frontend y CommonJS en backend.	Cumplido

Arquitectura del sistema

Justificación del patrón aplicado: Arquitectura en N – CAPAS, (N-Layer architecture), con separación de responsabilidades.

Dicha arquitectura permite separar responsabilidades, facilitando así el mantenimiento y la evolución independiente de cada uno de las capas, con docker compose se garantiza que cada capa sea un servicio completamente aislado, reemplazable y sobretodo escalable.

Capa	Tecnología	Responsabilidad
1. Presentación	React 18 + Vite + Zustand	Interfaz de usuario, gestión de estado de sesión, enrutamiento SPA.

2. Gateway / Proxy	Nginx	Proxy inverso, entrega de archivos estáticos, terminación TLS (producción).
3. Lógica de Negocio	Node.js + Express + Services/Controllers	API REST: validación, autenticación, reglas de negocio, autorización.
4. Acceso a Datos	Prisma ORM	Queries tipadas, migraciones, relaciones entre modelos.
5. Datos	PostgreSQL 16	Persistencia relacional de los 13 modelos del dominio.

Modelado de la base de datos

Modelo	Tabla	Descripción	# Campos
Usuario	usuarios	Entidad base de autenticación. Contiene nombre, teléfono (único), hash, rol y estado.	8
Productor	productores	Perfil extendido del productor: DPI, ubicación, hectáreas, descripción, calificación, estado de suspensión.	12
Comprador	compradores	Perfil extendido del comprador: razón social, NIT, tipo, ubicación, calificación.	9
Asociación	asociaciones	Perfil de asociación: nombre, NIT, ubicación, descripción, estado activa.	9
Producto	productos	Catálogo de productos agrícolas administrado por la asociación.	7
Publicación	publicaciones	Oferta de venta: título, cantidad, precio, ubicación, imagen, estado, fechas.	14
Negociación	negociaciones	Canal privado entre	10

		productor y comprador con estado, precio acordado y condiciones.	
Mensaje	mensajes	Mensajes de chat por negociación: contenido, remitente, estado leído.	6
Entrega	entregas	Registro de entrega asociado a negociación: fechas, lugar, estado.	8
ConfirmacionEntrega	confirmaciones_entrega	Confirmaciones individuales por usuario (doble confirmación).	6
Pago	pagos	Registro de pago: monto, método, referencia, estado, fecha.	9
Notificación	notificaciones	Alertas del sistema por usuario: tipo, título, mensaje, estado leído.	7
Calificación	calificaciones	Evaluaciones entre participantes de una negociación, con moderación de la asociación.	10

Endpoints para la API REST

Url base: /api/v1

Autenticación: bearer token (JWT)

Método	Ruta	Auth	Rol(es)	Descripción
POST	/auth/register	No	Todos	Registrar nuevo usuario (crea perfil por rol)
POST	/auth/login	No	Todos	Iniciar sesión, retorna JWT
GET	/auth/me	Sí	Todos	Datos del usuario autenticado con su

				perfil
PUT	/auth/password	Sí	Todos	Cambiar contraseña
GET	/publicaciones	No	Todos	Listar publicaciones activas (filtros: departamento, municipio, categoría)
GET	/publicaciones/mis-publicaciones	Sí	Productor	Publicaciones del productor autenticado
GET	/publicaciones/:id	No	Todos	Detalle de una publicación
POST	/publicaciones	Sí	Productor	Crear nueva publicación
PUT	/publicaciones/:id	Sí	Productor	Editar publicación propia
PATCH	/publicaciones/:id/estado	Sí	Productor	Cambiar estado (activa/pausada/cerrada)
DEL	/publicaciones/:id	Sí	Productor	Eliminar publicación propia
GET	/negociaciones	Sí	Prod./Comprador	Listar negociaciones del usuario
GET	/negociaciones/:id	Sí	Prod./Comprador	Detalle de negociación
POST	/negociaciones	Sí	Comprador	Crear negociación (desde publicación)
PATCH	/negociaciones/:id/estado	Sí	Prod./Comprador	Cambiar estado de negociación
GET	/negociaciones/:id/mensajes	Sí	Prod./Comprador	Listar mensajes de la negociación
POST	/negociaciones/:id/mensajes	Sí	Prod./Comprador	Enviar mensaje

			r	
PATCH	/negociaciones/:id/mensajes/leer	Sí	Prod./Comprador	Marcar mensajes como leídos
GET	/entregas	Sí	Prod./Comprador	Listar entregas del usuario
GET	/entregas/:id	Sí	Prod./Comprador	Detalle de entrega
POST	/entregas	Sí	Prod./Comprador	Crear entrega asociada a negociación
PUT	/entregas/:id	Sí	Prod./Comprador	Actualizar datos de entrega
POST	/entregas/:id/confirmar	Sí	Prod./Comprador	Confirmar recepción (doble confirmación)
GET	/pagos	Sí	Prod./Comprador	Listar pagos del usuario
GET	/pagos/:id	Sí	Prod./Comprador	Detalle de pago
POST	/pagos	Sí	Prod./Comprador	Registrar pago para negociación
PUT	/pagos/:id	Sí	Prod./Comprador	Actualizar estado/datos de pago
GET	/asociaciones	No	Todos	Listar asociaciones activas
POST	/asociaciones	Sí	Asociación	Crear perfil de asociación
GET	/asociaciones/	Sí	Asociación	Listar productores

	miembros			miembros
PUT	/asociaciones/ miembros/ suspender/:id	Sí	Asociación	Suspender productor (temporal/definitivo)
PUT	/asociaciones/ miembros/ reactivar/:id	Sí	Asociación	Reactivar productor suspendido
GET	/notificaciones	Sí	Todos	Listar notificaciones del usuario
PATCH	/notificaciones/:id/leer	Sí	Todos	Marcar una notificación como leída
PATCH	/notificaciones/leer- todas	Sí	Todos	Marcar todas las notificaciones como leídas
GET	/productos	No	Todos	Listar catálogo de productos
GET	/productos/:id	No	Todos	Detalle de producto
POST	/productos	Sí	Asociación	Crear producto en el catálogo
PUT	/productos/:id	Sí	Asociación	Editar producto del catálogo
GET	/dashboard	Sí	Todos	KPIs diferenciados por rol
POST	/uploads	Sí	Productor	Subir imagen de publicación
GET	/api/v1/health	No	Sistema	Health check del servidor

Modelo de datos (resumen)

El modelo relacional está definido en el esquema Prisma (schema.prisma) y se compone de las siguientes entidades. Todas se relacionan con el Usuario base a través del campo rol correspondiente.

Entidad Prisma	Descripción y relación con Usuario (rol)
Usuario	Cuenta base del sistema; contiene credenciales, rol y datos comunes.
Productor	Extiende Usuario; datos agrícolas del productor (ubicación, finca, etc.).
Comprador	Extiende Usuario; datos del comprador (entrega preferida, historial).
Asociación	Entidad organizadora que agrupa productores.
Producto	Catálogo de productos agrícolas ofertables.
Publicación	Oferta concreta de un producto por parte de un Productor o Asociación.
Negociación	Trato directo entre un Productor y un Comprador sobre una Publicación.
Mensaje	Mensaje de chat ligado a una Negociación.
Entrega	Registro físico de entrega ligado a una Negociación cerrada.
ConfirmacionEntrega	Doble confirmación (Productor y Comprador) de la Entrega.
Pago	Registro de pago asociado a una Negociación.
Notificación	Alerta automática emitida a un Usuario por eventos del sistema.
Calificación	Puntuación y reseña que un Usuario emite sobre otro tras una operación.

Equipo de trabajo y roles respectivos

Integrante	Rol técnico	Responsabilidad
Freddy Roberto Orozco Alvarado	Frontend Lead / Documentación	Diseño e implementación de la interfaz en React 18 + Vite + Zustand, rutas y componentes reutilizables.
Elmer Yovani Rodríguez de Paz	Backend Lead / Documentación	Diseño de la API REST en Express + Prisma, lógica de negocio, middlewares y seguridad (JWT, bcrypt).

Cronograma tentativo e hitos

Hito	Fecha	Descripción
Inicio	[20/02/2026]	Kick-off, conformación del equipo y del repositorio.
Análisis	[01/03/2026]	Levantamiento de requerimientos, casos de uso y modelo de datos.
Diseño	[15/03/2026]	Arquitectura, wireframes, contratos de API y diseño de BD.
Implementación	[21/04/2026]	Desarrollo iterativo de los siete módulos del sistema.
Pruebas	[20/05/2026]	Pruebas unitarias e integrales, corrección de defectos.
Cierre / Entrega	[04/06/2026]	Documentación final, manual de usuario, release notes y defensa.

Validación para la entregable

Criterios de aceptación

#	Criterio	Verificación	Estado
1	El sistema corre en contenedores Docker con docker compose up.	Ejecutar docker compose up y verificar 3 contenedores activos.	Verificado localmente
2	La aplicación es accesible desde el navegador en el VPS.	Abrir URL de producción en navegador.	[COMPLETAR: Pendiente VPS]
3	La aplicación opera bajo HTTPS con certificado válido.	Verificar candado SSL en navegador y certificado Let's Encrypt.	[COMPLETAR: Pendiente VPS]
4	El registro de usuarios funciona para los 3 roles.	Registrar un productor, un comprador y una asociación.	Verificado localmente
5	El login genera un JWT	Iniciar sesión y verificar	Verificado

	válido y el dashboard carga correctamente.	que el dashboard muestra KPIs correctos.	localmente
6	Un productor puede crear y gestionar publicaciones con imagen.	Crear publicación con foto, luego editarla y cambiar estado.	Verificado localmente
7	Un comprador puede iniciar una negociación y enviar mensajes.	Iniciar negociación desde publicación activa y enviar 2+ mensajes.	Verificado localmente
8	La confirmación doble de entregas funciona correctamente.	Confirmar entrega desde productor y desde comprador.	Verificado localmente
9	La asociación puede suspender y reactivar a un productor miembro.	Suspender un productor y verificar que no puede publicar. Reactivar.	Verificado localmente
10	La documentación de la API cubre todos los endpoints implementados.	Revisar Documentacion_API_LaEs peranza.docx contra lista de endpoints.	Verificado

Pruebas de aceptación de usuario (UAT)

Fecha de realización	[COMPLETAR: Fecha de UAT]
Lugar / modalidad	[COMPLETAR: Modalidad (presencial / remota)]
Evaluadores presentes	[COMPLETAR: Nombres de evaluadores (docente, cliente)]
Resultado general	[COMPLETAR: Resultado de la UAT]
Observaciones	[COMPLETAR: Observaciones registradas durante la UAT]
Conformidad firmada	[COMPLETAR: Acta de conformidad firmada adjunta]

Evidencias de pruebas

Las siguientes evidencias están disponibles en la carpeta /Documentación del repositorio:

- Evidencias_Cobertura_LaEsperanza.docx — reporte de cobertura de pruebas automatizadas
- Casos_Prueba_LaEsperanza.docx — 37 casos de prueba documentados con resultado
- Ejemplos_Pruebas_Automatizadas_LaEsperanza_CORREGIDO.test.js — código de pruebas con Jest + Supertest
- Capturas de pantalla del sistema en funcionamiento — [COMPLETAR: adjuntar]

Errores conocidos

ID	Descripción	Impacto	Estado
ERR-01	HTTPS no configurado en el docker-compose.yml actual. El sistema corre en HTTP.	Alto (requerimiento de producción)	Pendiente — configurar en VPS con Certbot
ERR-02	La UI de calificaciones no está implementada. El modelo y backend existen.	Bajo (funcionalidad futura)	Postergado a versión 1.1.0
ERR-03	Las notificaciones no se entregan en tiempo real (solo por polling/recarga).	Medio (experiencia de usuario)	Postergado a versión 1.1.0

Control de versiones del documento

Versión	Fecha	Autor	Cambios
1.0.0	[05/06/2026]	Freddy Orozco / Elmer Rodr7	Creación inicial del documento Datos Generales del Proyecto.

Fin del documento — Sistema La Esperanza, Datos Generales del Proyecto v1.0.0.